PART A

Experiment 1(a)

Question

Learn the use of basic UNIX commands – To access information using date, history, man, who, whoami, uptime, finger, cal.

Theory

UNIX provides a rich set of information-retrieval commands that allow users to query system status, user details, and scheduling information. These commands are fundamental to navigating and managing any UNIX/Linux environment. They run without arguments in most cases and return live system information.

Commands, Syntax, and Examples

Command: date

Syntax: date [+FORMAT]

```
student@ubuntu:~$ date
Thu Jul 15 23:50:40 IST 2026
```

Command: history

Syntax: history [n]

```
student@ubuntu:~$ history
    1  ls
    2  cd /home
    3  pwd
    4  date
```

Command: man

Syntax: man <command>

```
student@ubuntu:~$ man ls
(manual page for ls is displayed)
```

Command: who

Syntax: who [options]

```
student@ubuntu:~$ who
student  tty7  2026-07-15 09:00 (:0)
```

Command: whoami

Syntax: whoami

```
student@ubuntu:~$ whoami
student
```

Command: uptime

Syntax: uptime

```
student@ubuntu:~$ uptime
23:50:40 up 1:23,  1 user,  load average: 0.08, 0.03, 0.01
```

Command: finger

Syntax: finger [username]

```
student@ubuntu:~$ finger student
Login: student   Name: Student User
Directory: /home/student  Shell: /bin/bash
On since Thu Jul 15 09:00 (IST) on tty7
```

Command: cal

Syntax: cal [month] [year]

```
student@ubuntu:~$ cal
      July 2026
Su Mo Tu We Th Fr Sa
          1  2  3  4
 5  6  7  8  9 10 11
12 13 14 15 16 17 18
19 20 21 22 23 24 25
26 27 28 29 30 31
```

Result: The basic UNIX information commands (date, history, man, who, whoami, uptime, finger, cal) were executed successfully and produced the expected system information.

Experiment 1(b)

Question

Learn the use of basic UNIX commands – To display contents of files using cat, vi, more, head, tail, grep, cmp, wc.

Theory

UNIX provides powerful file-viewing and searching commands. These tools allow users to display, search, and compare file contents from the terminal. cat is used for concatenation and display, while grep performs pattern matching. head/tail display specific lines, and wc counts lines, words, and characters. cmp performs byte-level file comparison.

Commands, Syntax, and Examples

Command: cat — Display file contents

Syntax: cat [options] <file>

```
student@ubuntu:~$ cat file.txt
Hello World
This is UNIX Lab
Line Three
```

Command: vi — Full-screen text editor

Syntax: vi <filename>

```
student@ubuntu:~$ vi file.txt
(Opens vi editor in the terminal)
```

Command: more — Page-by-page file viewer

Syntax: more [options] <file>

```
student@ubuntu:~$ more file.txt
Hello World
This is UNIX Lab
--More--(50%)
```

Command: head — Show first N lines

Syntax: head -n <N> <file>

```
student@ubuntu:~$ head -3 file.txt
Hello World
This is UNIX Lab
Line Three
```

Command: tail — Show last N lines

Syntax: tail -n <N> <file>

```
student@ubuntu:~$ tail -2 file.txt
Second last line
Last line
```

Command: grep — Search for pattern

Syntax: grep [options] <pattern> <file>

```
student@ubuntu:~$ grep "UNIX" file.txt
This is UNIX Lab
```

Command: cmp — Compare two files byte-by-byte

Syntax: cmp <file1> <file2>

```
student@ubuntu:~$ cmp file1.txt file2.txt
file1.txt file2.txt differ: byte 6, line 1
```

Command: wc — Word/line/char count

Syntax: wc [options] <file>

```
student@ubuntu:~$ wc file.txt
  3   9  42 file.txt
```

Result: The file display commands (cat, vi, more, head, tail, grep, cmp, wc) were executed successfully on sample files.

Experiment 1(c)

Question

Learn the use of basic UNIX commands – To manage files using cat, cp, ls, mv, rm, chmod, find.

Theory

File management in UNIX is performed through a set of versatile commands. ls lists directory contents, cp copies files, mv moves or renames files, rm removes files, chmod changes file permissions, and find locates files based on given criteria. These are among the most frequently used UNIX utilities.

Commands, Syntax, and Examples

```
# List files in long format
student@ubuntu:~$ ls -la

# Create a file using cat
student@ubuntu:~$ cat > newfile.txt
Hello UNIX
^D

# Copy a file
student@ubuntu:~$ cp newfile.txt backup.txt

# Move/rename a file
student@ubuntu:~$ mv backup.txt archive.txt

# Change permissions (rwxr-xr-x)
student@ubuntu:~$ chmod 755 script.sh

# Remove a file
student@ubuntu:~$ rm archive.txt

# Find all .txt files recursively
student@ubuntu:~$ find . -name "*.txt"
```

Sample Output

```
student@ubuntu:~$ ls -la
total 32
drwxr-xr-x 2 student student 4096 Jul 15 23:50 .
drwxr-xr-x 8 student student 4096 Jul 15 09:00 ..
-rw-r--r-- 1 student student   11 Jul 15 23:45 newfile.txt
-rwxr-xr-x 1 student student  123 Jul 15 23:44 script.sh
student@ubuntu:~$ cp newfile.txt backup.txt
student@ubuntu:~$ mv backup.txt archive.txt
student@ubuntu:~$ chmod 755 script.sh
student@ubuntu:~$ rm archive.txt
student@ubuntu:~$ find . -name "*.txt"
./newfile.txt
student@ubuntu:~$
```

Result: The file management commands (cat, cp, ls, mv, rm, chmod, find) were executed successfully and produced the expected results.

Experiment 1(d)

Question

Learn the use of basic UNIX commands – Process utilities using ps, pid, ppid, tty, time, kill, exit.

Theory

UNIX process management commands allow users to monitor and control running processes. ps displays information about active processes. Every process has a unique PID (Process ID) and a PPID (Parent Process ID). The tty command shows the terminal associated with the current session. kill sends signals to processes, time measures execution duration, and exit terminates the current shell.

Commands, Syntax, and Examples

```
# Display all running processes
student@ubuntu:~$ ps -ef

# Show process id of current shell
student@ubuntu:~$ echo $$

# Show parent process id
student@ubuntu:~$ echo $PPID

# Show terminal name
student@ubuntu:~$ tty

# Measure execution time
student@ubuntu:~$ time ls

# Kill a process by PID
student@ubuntu:~$ kill -9 <PID>

# Exit the shell
student@ubuntu:~$ exit
```

Sample Output

```
student@ubuntu:~$ ps -ef
UID        PID  PPID  C STIME TTY      TIME     CMD
root         1     0  0 09:00 ?        00:00:02 /sbin/init
student   1234   567  0 09:05 pts/0    00:00:00 bash
student   1289  1234  0 23:50 pts/0    00:00:00 ps -ef

student@ubuntu:~$ echo $$
1234

student@ubuntu:~$ echo $PPID
567

student@ubuntu:~$ tty
/dev/pts/0

student@ubuntu:~$ time ls
file.txt  script.sh
real    0m0.003s
user    0m0.001s
sys     0m0.002s
student@ubuntu:~$
```

Result: Process utilities (ps, pid, ppid, tty, time, kill, exit) were demonstrated and produced the expected system outputs.

Experiment 1(e)

Question

Learn the use of basic UNIX commands – Directory handling utilities using cd, mkdir, rmdir, mv, pwd.

Theory

Directory management in UNIX is essential for organising files in a hierarchical file system. pwd prints the current working directory, mkdir creates new directories, cd changes the current directory, mv can rename or move directories, and rmdir removes empty directories. These commands form the backbone of filesystem navigation in UNIX/Linux.

Commands, Syntax, and Examples

```
# Print working directory
student@ubuntu:~$ pwd

# Create a new directory
student@ubuntu:~$ mkdir mydir

# Change into the new directory
student@ubuntu:~$ cd mydir

# Go back to parent directory
student@ubuntu:~/mydir$ cd ..

# Rename/move a directory
student@ubuntu:~$ mv mydir newdir

# Remove the directory
student@ubuntu:~$ rmdir newdir
```

Sample Output

```
student@ubuntu:~$ pwd
/home/student
student@ubuntu:~$ mkdir mydir
student@ubuntu:~$ cd mydir
student@ubuntu:~/mydir$ pwd
/home/student/mydir
student@ubuntu:~/mydir$ cd ..
student@ubuntu:~$ mv mydir newdir
student@ubuntu:~$ ls
newdir  file.txt  script.sh
student@ubuntu:~$ rmdir newdir
student@ubuntu:~$ ls
file.txt  script.sh
student@ubuntu:~$
```

Result: Directory handling utilities (cd, mkdir, rmdir, mv, pwd) were executed successfully and produced the expected outputs.

Experiment 2

Question

Write a shell script that displays list of all the files in the current directory to which the user has read, write and execute permissions.

Theory

In UNIX, every file has three sets of permissions: read (r), write (w), and execute (x). The shell provides built-in test operators -r, -w, and -x to check whether these permissions are set for the current user. A shell script can iterate over directory entries and use these tests to filter files that match all three conditions.

Algorithm

- Start.
- Use a for loop to iterate over all files in the current directory.
- For each entry, check if it is a regular file using [-f].
- Check read, write, and execute permissions using -r, -w, -x test operators.
- If all three conditions are true, print the filename.
- Stop.

Shell Script

```
#!/bin/bash
# Script: exp2_rwx.sh
# Purpose: List files with read, write, and execute permissions

echo "Files with Read, Write, and Execute permissions:"
echo "================================================="

found=0
for file in *
do
    if [ -f "$file" ] && [ -r "$file" ] && [ -w "$file" ] && [ -x
"$file" ]
    then
        echo "$file"
        found=1
    fi
done

if [ $found -eq 0 ]; then
    echo "No files with rwx permissions found."
fi
```

Code Explanation

The for loop iterates over all entries in the current directory. The [-f] test ensures only regular files are considered. The -r, -w, and -x operators test read, write, and execute permissions respectively for the current user. The found flag tracks whether any matching files were discovered.

Execution

```
student@ubuntu:~$ chmod +x exp2_rwx.sh
student@ubuntu:~$ ./exp2_rwx.sh
```

Sample Output

```
student@ubuntu:~$ ./exp2_rwx.sh
Files with Read, Write, and Execute permissions:
==================================================
exp2_rwx.sh
run_demo.sh
student@ubuntu:~$
```

Result: The shell script successfully listed all files in the current directory for which the user has read, write, and execute permissions.

Experiment 3

Question

Write a shell script that accepts a list of file names as its arguments, count and reports the occurrence of each word that is present in the first argument file on other argument files.

Theory

The script reads all words from the first file argument and for each word counts how many times it appears in the remaining files. This uses grep -ow to count exact whole-word occurrences, wc -l to total the count, and a while loop to iterate over each word in the reference file.

Algorithm

- Start.
- Validate that at least 2 arguments are provided.
- Store the first argument as the word-reference file; shift to get the remaining files.
- Read each word from the reference file using a while read loop.
- For each word, use grep -ow across all remaining files and pipe to wc -l for count.
- Print the word and its total count.
- Stop.

Shell Script

```
#!/bin/bash
# Script: exp3_wordcount.sh
# Purpose: Count occurrence of words from file1 in remaining files

if [ $# -lt 2 ]; then
    echo "Usage: $0 <word_file> <file1> [file2] ..."
    exit 1
fi

wordfile="$1"
shift

if [ ! -f "$wordfile" ]; then
    echo "Error: Word file '$wordfile' not found."
    exit 1
fi

echo "Word occurrence report (reference: $wordfile)"
echo "================================================"
printf "%-20s %s\n" "Word" "Count"
echo "------------------------------"

while IFS= read -r word; do
    # Skip empty lines
    [ -z "$word" ] && continue

    count=0
    for file in "$@"; do
        if [ -f "$file" ]; then
            c=$(grep -ow "$word" "$file" 2>/dev/null | wc -l)
            count=$((count + c))
        fi
```

```
    done
    printf "%-20s %d\n" "$word" "$count"
done < "$wordfile"
```

Code Explanation

After argument validation, the wordfile is stored and remaining args shift to "$@". The while IFS= read loop reads line-by-line from the word file. grep -ow finds only exact whole-word matches (case-sensitive); wc -l counts them. Counts across all files are summed and printed.

Execution

```
student@ubuntu:~$ cat words.txt
hello
UNIX
student@ubuntu:~$ chmod +x exp3_wordcount.sh
student@ubuntu:~$ ./exp3_wordcount.sh words.txt file1.txt file2.txt
```

Sample Output

```
student@ubuntu:~$ ./exp3_wordcount.sh words.txt file1.txt file2.txt
Word occurrence report (reference: words.txt)
================================================
Word                 Count
----------------------------
hello                3
UNIX                 5
student@ubuntu:~$
```

Result: The shell script successfully counted and reported the occurrence of each word from the reference file across all other argument files.

Experiment 4

Question

Write a shell script that accepts one or more file name as arguments and converts all of them to uppercase, provided they exist in the current directory.

Theory

The script iterates over each filename argument and checks whether the file exists in the current directory using the -f test operator. The tr command with the [:lower:] and [:upper:] character class translators converts the filename string to uppercase. The mv command then renames the file to the uppercase version.

Algorithm

- Start.
- Check that at least one argument is provided.
- For each argument, verify the file exists in the current directory.
- Convert the filename to uppercase using tr ['a-z'] ['A-Z'].
- If the new name differs from the original, rename using mv.
- Print a confirmation message for each renamed file.
- Stop.

Shell Script

```
#!/bin/bash
# Script: exp4_uppercase.sh
# Purpose: Convert file names to uppercase

if [ $# -eq 0 ]; then
    echo "Usage: $0 <file1> [file2] ..."
    exit 1
fi

for filename in "$@"
do
    if [ -f "$filename" ]; then
        newname=$(echo "$filename" | tr '[:lower:]' '[:upper:]')
        if [ "$filename" = "$newname" ]; then
            echo "[$filename] is already uppercase. Skipping."
        else
            mv "$filename" "$newname"
            echo "Renamed: $filename  -->  $newname"
        fi
    else
        echo "Warning: File '$filename' does not exist in the current
directory."
    fi
done
```

Code Explanation

The for loop processes each command-line argument. [-f "$filename"] confirms the file is a regular file in the current directory. tr '[:lower:]' '[:upper:]' translates lowercase letters to uppercase. If the new name differs from the original, mv renames the file. Files already in uppercase are skipped gracefully.

Execution

```
student@ubuntu:~$ ls
hello.txt  notes.txt  README.TXT
student@ubuntu:~$ chmod +x exp4_uppercase.sh
student@ubuntu:~$ ./exp4_uppercase.sh hello.txt notes.txt README.TXT
```

Sample Output

```
student@ubuntu:~$ ./exp4_uppercase.sh hello.txt notes.txt README.TXT
Renamed: hello.txt  -->  HELLO.TXT
Renamed: notes.txt  -->  NOTES.TXT
[README.TXT] is already uppercase. Skipping.
student@ubuntu:~$ ls
HELLO.TXT  NOTES.TXT  README.TXT
student@ubuntu:~$
```

Result: The shell script successfully converted all existing filenames from lowercase to uppercase using the tr utility.

Experiment 5

Question

Write grep commands to the following: a. To select the lines from a file that has exactly 2 characters. b. To select the lines from a file that has more than one blank space.

Theory

grep (Global Regular Expression Print) is a powerful UNIX command for searching text using regular expressions (regex). The -E option enables Extended Regular Expressions (ERE). Anchors ^ and $ mark line start and end. The dot (.) matches any character. A space literal inside a regex matches blank spaces. + is a quantifier meaning "one or more".

Part a: Select Lines with Exactly 2 Characters

Regular expression: ^..$ (^ marks start, .. matches any two characters, $ marks end)

```
grep -E "^..$" filename.txt
```

Alternatively using BRE:

```
grep "^..$" filename.txt
```

Sample Input — sample.txt

```
Hi
Hello
OK
A
No
Yes
Go
```

Sample Output

```
student@ubuntu:~$ grep -E "^..$" sample.txt
Hi
OK
No
Go
student@ubuntu:~$
```

Part b: Select Lines with More Than One Blank Space

Regular expression: " +" — matches two or more consecutive spaces (one space followed by one-or-more spaces). Alternatively use \s{2,} with ERE.

```
# Method 1: Two literal spaces followed by zero-or-more spaces
grep "  \+" filename.txt

# Method 2: Using ERE
grep -E "  +" filename.txt

# Method 3: Using character class
grep -E "[[:blank:]]{2,}" filename.txt
```

Sample Input — spaces.txt

```
This line has one space only.
This  line  has  extra  spaces.
Normal line here.
Two  spaces  here.
```

Sample Output

```
student@ubuntu:~$ grep -E "  +" spaces.txt
This  line  has  extra  spaces.
Two  spaces  here.
student@ubuntu:~$
```

Result: The grep commands successfully selected lines with exactly 2 characters and lines containing more than one consecutive blank space.

Experiment 6

Question

Write a shell script which accepts two file names as arguments. Compare the contents. If they are same, then delete the second file.

Theory

The cmp command compares two files byte by byte. When used with the -s (silent) flag, it suppresses all output and returns exit status 0 if files are identical, and non-zero otherwise. This allows shell scripts to branch on the comparison result. If files match, rm is used to delete the second file.

Algorithm

- Start.
- Validate that exactly two arguments are provided.
- Check both files exist using the -f test operator.
- Use cmp -s to silently compare both files.
- If exit status is 0 (files identical), delete the second file using rm.
- If exit status is non-zero, print "Files are different".
- Stop.

Shell Script

```
#!/bin/bash
# Script: exp6_compare.sh
# Purpose: Compare two files; delete second if identical

if [ $# -ne 2 ]; then
    echo "Usage: $0 <file1> <file2>"
    exit 1
fi

file1="$1"
file2="$2"

# Check if both files exist
if [ ! -f "$file1" ]; then
    echo "Error: File '$file1' not found."
    exit 1
fi

if [ ! -f "$file2" ]; then
    echo "Error: File '$file2' not found."
    exit 1
fi

# Compare the files silently
if cmp -s "$file1" "$file2"; then
    echo "Files '$file1' and '$file2' are IDENTICAL."
    rm "$file2"
    echo "'$file2' has been deleted."
else
    echo "Files '$file1' and '$file2' are DIFFERENT. No action taken."
fi
```

Code Explanation

Argument validation ensures exactly two filenames are passed. The -f test confirms each file exists. cmp -s performs a silent byte-level comparison; its exit code (0 = same, 1 = different) drives the if-else decision. On match, rm deletes the second file.

Execution

```
student@ubuntu:~$ echo "Hello World" > file1.txt
student@ubuntu:~$ cp file1.txt file2.txt
student@ubuntu:~$ chmod +x exp6_compare.sh
student@ubuntu:~$ ./exp6_compare.sh file1.txt file2.txt
```

Sample Output

```
student@ubuntu:~$ ./exp6_compare.sh file1.txt file2.txt
Files 'file1.txt' and 'file2.txt' are IDENTICAL.
'file2.txt' has been deleted.

student@ubuntu:~$ ./exp6_compare.sh file1.txt other.txt
Files 'file1.txt' and 'other.txt' are DIFFERENT. No action taken.
student@ubuntu:~$
```

Result: The shell script successfully compared two files and deleted the second file when their contents were found to be identical.

Experiment 7

Question

Write a shell script: a. To count number of lines in a file that do not contain vowels. b. To count number of characters, words, lines in a given file.

Theory

grep with the -v flag inverts the match, selecting lines that do NOT match the given pattern. The character class [aeiouAEIOU] matches any vowel. Piping to wc -l counts the resulting lines. The wc command (word count) directly provides line, word, and character statistics via the -l, -w, and -c flags respectively.

Part a – Count Lines Without Vowels

Algorithm

- Start.
- Validate that one argument (filename) is provided.
- Verify the file exists.
- Use grep -cv "[aeiouAEIOU]" to count non-matching lines.
- Display the count.
- Stop.

Shell Script — Part a

```
#!/bin/bash
# Script: exp7a_novowels.sh
# Purpose: Count lines that do not contain vowels

if [ $# -ne 1 ]; then
    echo "Usage: $0 <filename>"
    exit 1
fi

file="$1"

if [ ! -f "$file" ]; then
    echo "Error: File '$file' not found."
    exit 1
fi

count=$(grep -cv "[aeiouAEIOU]" "$file")
echo "Number of lines without vowels in '$file': $count"
```

Execution and Output — Part a

```
student@ubuntu:~$ ./exp7a_novowels.sh data.txt
student@ubuntu:~$ ./exp7a_novowels.sh data.txt
Number of lines without vowels in 'data.txt': 3
student@ubuntu:~$
```

Part b – Count Characters, Words, Lines

Shell Script — Part b

```
#!/bin/bash
# Script: exp7b_wc.sh
```

```
# Purpose: Count characters, words, and lines in a file

if [ $# -ne 1 ]; then
    echo "Usage: $0 <filename>"
    exit 1
fi

file="$1"

if [ ! -f "$file" ]; then
    echo "Error: File '$file' not found."
    exit 1
fi

lines=$(wc -l < "$file")
words=$(wc -w < "$file")
chars=$(wc -c < "$file")

echo "File     : $file"
echo "Lines    : $lines"
echo "Words    : $words"
echo "Characters: $chars"
```

Execution and Output — Part b

```
student@ubuntu:~$ ./exp7b_wc.sh data.txt
student@ubuntu:~$ ./exp7b_wc.sh data.txt
File     : data.txt
Lines    : 8
Words    : 34
Characters: 182
student@ubuntu:~$
```

Result: Both shell scripts successfully counted (a) lines without vowels using grep -cv and (b) characters, words, and lines using the wc command.

Experiment 8

Question

Write a shell script to list all the files in a given directory.

Theory

A shell script can iterate over the contents of any directory by using a glob pattern with a for loop. The directory path is passed as a command-line argument. The [-f] test operator distinguishes regular files from directories and other special entries. basename extracts just the filename from the full path.

Algorithm

- Start.
- Validate that one directory path argument is provided.
- Verify the path is a valid directory using [-d].
- Iterate over all entries in the directory using a for loop with glob.
- Print only regular files ([-f] test).
- Print total count at end.
- Stop.

Shell Script

```
#!/bin/bash
# Script: exp8_listfiles.sh
# Purpose: List all files in a given directory

if [ $# -ne 1 ]; then
    echo "Usage: $0 <directory>"
    exit 1
fi

dir="$1"

if [ ! -d "$dir" ]; then
    echo "Error: '$dir' is not a valid directory."
    exit 1
fi

echo "Files in directory: $dir"
echo "================================"

count=0
for item in "$dir"/*
do
    if [ -f "$item" ]; then
        echo "  $(basename "$item")"
        count=$((count + 1))
    fi
done

echo "--------------------------------"
echo "Total files: $count"
```

Code Explanation

The directory path is passed as $1. The glob "$dir"/* expands to all entries. The -f test filters out subdirectories and special files. basename strips the directory prefix for clean output. The count variable is incremented for each file found.

Execution

```
student@ubuntu:~$ chmod +x exp8_listfiles.sh
student@ubuntu:~$ ./exp8_listfiles.sh /home/student
```

Sample Output

```
student@ubuntu:~$ ./exp8_listfiles.sh /home/student
Files in directory: /home/student
==============================
  exp2_rwx.sh
  exp3_wordcount.sh
  exp8_listfiles.sh
  file.txt
  notes.txt
------------------------------
Total files: 5
student@ubuntu:~$
```

Result: The shell script successfully listed all regular files in the specified directory along with the total file count.

Experiment 9

Question

Write a shell script to display list of users currently logged in.

Theory

The who command reads the /var/run/utmp file and prints a list of all users currently logged into the system, along with their terminal and login time. The w command provides additional information including CPU usage. Piping the output of who to wc -l gives the total count of logged-in users.

Algorithm

- Start.
- Print a header.
- Execute who to display all currently logged-in users.
- Count total users by piping who output to wc -l.
- Display the count.
- Stop.

Shell Script

```
#!/bin/bash
# Script: exp9_loggedusers.sh
# Purpose: Display list of users currently logged in

echo "Currently Logged-in Users"
echo "========================="
echo ""

printf "%-12s %-10s %-20s %s\n" "USER" "TTY" "LOGIN TIME" "FROM"
echo "-------------------------------------------------------"

who | while read user tty logintime rest; do
    printf "%-12s %-10s %-20s %s\n" "$user" "$tty" "$logintime" "$rest"
done

echo "-------------------------------------------------------"
total=$(who | wc -l)
echo "Total users logged in: $total"

# Additional: show current user
echo ""
echo "You are logged in as: $(whoami)"
```

Code Explanation

who outputs login details. A while read loop formats each user's information with aligned columns. wc -l on the who output counts total sessions. whoami confirms the current user identity.

Execution

```
student@ubuntu:~$ chmod +x exp9_loggedusers.sh
student@ubuntu:~$ ./exp9_loggedusers.sh
```

Sample Output

```
student@ubuntu:~$ ./exp9_loggedusers.sh
```

```
Currently Logged-in Users
=========================

USER         TTY        LOGIN TIME           FROM
------------------------------------------------------------
student      tty7       2026-07-15 09:00     (:0)
student      pts/0      2026-07-15 09:10     (:0.0)
------------------------------------------------------------
Total users logged in: 2

You are logged in as: student
student@ubuntu:~$
```

Result: The shell script successfully displayed the list of users currently logged into the system along with their login details and total count.

Experiment 10

Question

Write a shell script to read three text files in the current directory and merge them into a single file and returns a file descriptor for the new file.

Theory

Shell scripts can manipulate file descriptors using the exec built-in. File descriptors 0, 1, and 2 are reserved for stdin, stdout, and stderr respectively. Descriptors 3 and above can be opened using exec N<file or exec N>file. The cat command with multiple arguments concatenates files and the output redirection (>) creates or overwrites the target file.

Algorithm

- Start.
- Validate that exactly three file arguments are provided.
- Verify all three files exist.
- Use cat to concatenate all three files into a merged output file.
- Open the merged file and assign it to file descriptor 3 using exec.
- Display the contents of the merged file.
- Close the file descriptor.
- Stop.

Shell Script

```
#!/bin/bash
# Script: exp10_merge.sh
# Purpose: Merge three text files and return a file descriptor

if [ $# -ne 3 ]; then
    echo "Usage: $0 <file1> <file2> <file3>"
    exit 1
fi

file1="$1"
file2="$2"
file3="$3"
merged_file="merged_output.txt"

# Check all files exist
for f in "$file1" "$file2" "$file3"; do
    if [ ! -f "$f" ]; then
        echo "Error: File '$f' not found."
        exit 1
    fi
done

# Merge all three files into one
cat "$file1" "$file2" "$file3" > "$merged_file"
echo "Files merged successfully into: $merged_file"

# Open the merged file and assign to file descriptor 3
exec 3< "$merged_file"
echo "File descriptor 3 assigned to: $merged_file"
```

```
echo ""
echo "Contents of merged file:"
echo "========================"

# Read from file descriptor 3 and display
while IFS= read -r line <&3; do
    echo "$line"
done

# Close the file descriptor
exec 3<&-
echo ""
echo "File descriptor 3 closed."
```

Code Explanation

cat concatenates all three files and writes to merged_output.txt. exec 3< "$merged_file" opens the merged file for reading on file descriptor 3. The while IFS= read -r line <&3 loop reads from FD 3 line by line and prints each line. exec 3<&- closes the file descriptor when done.

Execution

```
student@ubuntu:~$ cat file1.txt
Contents of File 1
student@ubuntu:~$ cat file2.txt
Contents of File 2
student@ubuntu:~$ cat file3.txt
Contents of File 3
student@ubuntu:~$ chmod +x exp10_merge.sh
student@ubuntu:~$ ./exp10_merge.sh file1.txt file2.txt file3.txt
```

Sample Output

```
student@ubuntu:~$ ./exp10_merge.sh file1.txt file2.txt file3.txt
Files merged successfully into: merged_output.txt
File descriptor 3 assigned to: merged_output.txt

Contents of merged file:
========================
Contents of File 1
Contents of File 2
Contents of File 3

File descriptor 3 closed.
student@ubuntu:~$
```

Result: The shell script successfully merged three text files into a single file and demonstrated file descriptor assignment and I/O redirection using exec.

PART B

Experiment 1

Question

Write a program to copy a file into another using system calls.

Theory

UNIX system calls provide low-level file I/O. open() opens a file and returns a file descriptor. read() reads bytes from a file descriptor into a buffer. write() writes bytes from a buffer to a file descriptor. close() releases the file descriptor. The O_RDONLY, O_WRONLY, O_CREAT, and O_TRUNC flags control file opening mode.

Algorithm

- Start.
- Accept source and destination filenames as command-line arguments.
- Open source file with O_RDONLY using open().
- Create/open destination file with O_WRONLY | O_CREAT | O_TRUNC using open().
- Repeatedly call read() into a buffer until EOF (returns 0).
- Write the buffer contents to the destination using write().
- Close both file descriptors.
- Stop.

C Program

```
/* filecopy.c - Copy a file using UNIX system calls */
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>

#define BUFFER_SIZE 1024

int main(int argc, char *argv[]) {
    int src_fd, dest_fd;
    char buffer[BUFFER_SIZE];
    ssize_t bytes_read, bytes_written;

    if (argc != 3) {
        fprintf(stderr, "Usage: %s <source> <destination>\n", argv[0]);
        exit(EXIT_FAILURE);
    }

    /* Open source file for reading */
    src_fd = open(argv[1], O_RDONLY);
    if (src_fd == -1) {
        perror("Error opening source file");
        exit(EXIT_FAILURE);
    }

    /* Create/open destination file for writing */
    dest_fd = open(argv[2], O_WRONLY | O_CREAT | O_TRUNC, 0644);
    if (dest_fd == -1) {
        perror("Error opening destination file");
```

```
        close(src_fd);
        exit(EXIT_FAILURE);
    }

    /* Copy: read from source, write to destination */
    while ((bytes_read = read(src_fd, buffer, BUFFER_SIZE)) > 0) {
        bytes_written = write(dest_fd, buffer, bytes_read);
        if (bytes_written != bytes_read) {
            perror("Write error");
            close(src_fd);
            close(dest_fd);
            exit(EXIT_FAILURE);
        }
    }

    if (bytes_read == -1)
        perror("Read error");
    else
        printf("File '%s' copied to '%s' successfully.\n", argv[1],
argv[2]);

    close(src_fd);
    close(dest_fd);
    return 0;
}
```

Code Explanation

open() is called with O_RDONLY for the source, and O_WRONLY | O_CREAT | O_TRUNC with permission mode 0644 for the destination. The while loop reads up to BUFFER_SIZE bytes at a time. A return of 0 from read() signals EOF. bytes_written is verified to equal bytes_read to detect partial writes.

Compilation

```
student@ubuntu:~$ gcc filecopy.c -o filecopy
```

Execution

```
student@ubuntu:~$ echo "Hello UNIX Lab" > source.txt
student@ubuntu:~$ ./filecopy source.txt destination.txt
```

Sample Output

```
student@ubuntu:~$ ./filecopy source.txt destination.txt
File 'source.txt' copied to 'destination.txt' successfully.
student@ubuntu:~$ cat destination.txt
Hello UNIX Lab
student@ubuntu:~$
```

Result: The C program successfully copied a file using UNIX system calls open(), read(), write(), and close().

Experiment 2

Question

Write a program using system call: create, open, write, close, stat, fstat, lseek.

Theory

UNIX system calls provide direct kernel-level file access. creat() creates a new file (equivalent to open with O_CREAT|O_WRONLY|O_TRUNC). open() returns a file descriptor. write() writes data, read() reads data. lseek() repositions the file offset. stat() retrieves file metadata by filename; fstat() does the same using a file descriptor. close() frees the descriptor.

Algorithm

- Create file with creat() and write data using write().
- Close the file, then reopen with open() for reading.
- Use lseek() to reposition the read offset to the start.
- Read the data back with read() and display.
- Retrieve file metadata with fstat() using the open descriptor.
- Close the file and retrieve metadata again with stat() using filename.
- Stop.

C Program

```
/* syscalls_demo.c - Demonstrate UNIX file system calls */
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/stat.h>
#include <string.h>

#define FILENAME "syscall_demo.txt"

int main() {
    int fd;
    char write_buf[] = "Hello, UNIX System Calls Demo!\n";
    char read_buf[100];
    struct stat file_info;
    ssize_t bytes;

    /* --- creat() --- */
    fd = creat(FILENAME, 0644);
    if (fd == -1) { perror("creat"); exit(1); }
    printf("1. creat(): File created. fd = %d\n", fd);

    /* --- write() --- */
    bytes = write(fd, write_buf, strlen(write_buf));
    printf("2. write(): %ld bytes written.\n", bytes);

    /* --- close() --- */
    close(fd);
    printf("3. close(): File descriptor closed.\n");

    /* --- open() --- */
    fd = open(FILENAME, O_RDONLY);
```

```
    if (fd == -1) { perror("open"); exit(1); }
    printf("4. open(): File opened for reading. fd = %d\n", fd);

    /* --- lseek() --- */
    off_t offset = lseek(fd, 0, SEEK_SET);
    printf("5. lseek(): Offset moved to position %ld (SEEK_SET).\n",
offset);

    /* --- read() --- */
    memset(read_buf, 0, sizeof(read_buf));
    bytes = read(fd, read_buf, sizeof(read_buf) - 1);
    printf("6. read(): %ld bytes read.\n", bytes);
    printf("   Content: %s", read_buf);

    /* --- fstat() --- */
    if (fstat(fd, &file_info) == -1) { perror("fstat"); exit(1); }
    printf("7. fstat(): Size = %ld bytes, Permissions = %04o\n",
           (long)file_info.st_size, file_info.st_mode & 0777);

    close(fd);

    /* --- stat() --- */
    if (stat(FILENAME, &file_info) == -1) { perror("stat"); exit(1); }
    printf("8. stat():  Inode = %lu, Hard links = %lu\n",
           (unsigned long)file_info.st_ino,
           (unsigned long)file_info.st_nlink);

    remove(FILENAME);
    return 0;
}
```

Code Explanation

creat() creates the file with permissions 0644 and returns a write-only FD. write() writes the string. After close() and open() (read-only), lseek() with SEEK_SET repositions the offset to byte 0. fstat() uses the open FD to query file metadata. stat() uses the filename instead and returns inode and link count.

Compilation

```
student@ubuntu:~$ gcc syscalls_demo.c -o syscalls_demo
```

Execution

```
student@ubuntu:~$ ./syscalls_demo
```

Sample Output

```
student@ubuntu:~$ ./syscalls_demo
1. creat(): File created. fd = 3
2. write(): 31 bytes written.
3. close(): File descriptor closed.
4. open(): File opened for reading. fd = 3
5. lseek(): Offset moved to position 0 (SEEK_SET).
6. read(): 31 bytes read.
   Content: Hello, UNIX System Calls Demo!
7. fstat(): Size = 31 bytes, Permissions = 0644
8. stat():  Inode = 1572867, Hard links = 1
student@ubuntu:~$
```

Result: The C program successfully demonstrated all required UNIX system calls: creat, open, write, close, lseek, fstat, and stat.

Experiment 3

Question

Write a program to create a child process and allow the parent to display "parent" and the child to display "child" on the screen.

Theory

fork() is a UNIX system call that creates a new process by duplicating the calling process. After fork(), two processes run concurrently. fork() returns 0 in the child process, the child's PID in the parent, and -1 on failure. wait() in the parent blocks until the child terminates, preventing zombie creation.

Process Diagram

```
          main()
            |
          fork()
         /      \
    pid > 0      pid == 0
   (Parent)      (Child)
       |             |
 print "parent"  print "child"
       |             |
    wait()          exit()
       |
     exit()
```

Algorithm

- Call fork() to create a child process.
- If fork() returns < 0: error – exit.
- If fork() returns 0: child – print "child" and exit.
- If fork() returns > 0: parent – print "parent", then call wait().
- Stop.

C Program

```
/* fork_demo.c - Parent prints "parent", child prints "child" */
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    pid_t pid;

    printf("Creating child process using fork()...\n");

    pid = fork();

    if (pid < 0) {
        /* fork failed */
        perror("fork failed");
        exit(EXIT_FAILURE);

    } else if (pid == 0) {
        /* Child process */
```

```
        printf("child\n");
        printf("[Child]  PID = %d, PPID = %d\n", getpid(), getppid());
        exit(EXIT_SUCCESS);

    } else {
        /* Parent process */
        printf("parent\n");
        printf("[Parent] PID = %d, Child PID = %d\n", getpid(), pid);
        wait(NULL);   /* Wait for child to terminate */
        printf("Child process completed.\n");
    }

    return 0;
}
```

Code Explanation

fork() duplicates the process. The return value determines the execution branch. The child uses getpid() for its own PID and getppid() to retrieve the parent's PID. The parent calls wait(NULL) to block until the child finishes, ensuring no zombie is left.

Compilation

```
student@ubuntu:~$ gcc fork_demo.c -o fork_demo
```

Execution

```
student@ubuntu:~$ ./fork_demo
```

Sample Output

```
student@ubuntu:~$ ./fork_demo
Creating child process using fork()...
parent
[Parent] PID = 4821, Child PID = 4822
child
[Child]  PID = 4822, PPID = 4821
Child process completed.
student@ubuntu:~$
```

Result: The C program successfully created a child process using fork(); the parent displayed "parent" and the child displayed "child" on the screen.

Experiment 4

Question

Write a program to create a Zombie process.

Theory

A zombie (defunct) process is a child process that has completed execution but still has an entry in the process table because the parent has not yet called wait() to read its exit status. The child occupies a process table slot in the Z (zombie) state. Zombies are eventually cleaned up by the init process when the parent exits.

Zombie Process Lifecycle

```
  Parent (fork) -----> Child
                         |
                    child exits
                         |
                 [ZOMBIE STATE: Z]
  Parent does NOT call wait()
                         |
  ps aux | grep Z  <-- Zombie visible here
                         |
  Parent eventually exits
                         |
  init() adopts and reaps zombie
```

Algorithm

- Call fork() to create a child process.
- Child process prints its PID and exits immediately.
- Parent does NOT call wait() – child becomes a zombie.
- Parent sleeps for 10 seconds (zombie visible during this time).
- Parent exits, init reaps the zombie.

C Program

```
/* zombie.c - Create a zombie process */
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int main() {
    pid_t pid;

    printf("Demonstrating Zombie Process Creation\n");

    pid = fork();

    if (pid < 0) {
        perror("fork failed");
        exit(EXIT_FAILURE);

    } else if (pid == 0) {
        /* Child process: exits immediately */
        printf("[Child]  PID = %d  -> Exiting now (becomes zombie).\n",
               getpid());
```

```
        exit(EXIT_SUCCESS);

    } else {
        /* Parent: does NOT call wait() */
        printf("[Parent] PID = %d, Child PID = %d\n", getpid(), pid);
        printf("[Parent] Sleeping for 10 seconds without calling
wait().\n");
        printf("[Parent] Run: ps aux | grep Z  to see the zombie.\n");
        sleep(10);
        printf("[Parent] Exiting. init will clean up the zombie.\n");
    }

    return 0;
}
```

Code Explanation

After fork(), the child calls exit() immediately. The parent deliberately avoids wait(), so the child's PCB entry remains in the process table in the Z (zombie) state. During the 10-second sleep, running ps aux | grep Z in another terminal will show the zombie. When the parent exits, init() reaps it.

Compilation

```
student@ubuntu:~$ gcc zombie.c -o zombie
```

Execution

```
student@ubuntu:~$ ./zombie &
# In another terminal:
student@ubuntu:~$ ps aux | grep Z
```

Sample Output

```
student@ubuntu:~$ ./zombie
Demonstrating Zombie Process Creation
[Parent] PID = 5100, Child PID = 5101
[Parent] Sleeping for 10 seconds without calling wait().
[Parent] Run: ps aux | grep Z  to see the zombie.
[Child]  PID = 5101  -> Exiting now (becomes zombie).

# In another terminal during 10-second window:
student@ubuntu:~$ ps aux | grep Z
student   5101   5100  0 23:50 pts/0  00:00:00 [zombie] <defunct>

[Parent] Exiting. init will clean up the zombie.
student@ubuntu:~$
```

Result: The C program successfully created a zombie process by having the child exit without the parent calling wait(), demonstrating the defunct process state.

Experiment 5

Question

Write a program to implement inter process communication using pipes.

Theory

A pipe is a unidirectional inter-process communication channel. The pipe() system call creates two file descriptors: pipefd[0] (read end) and pipefd[1] (write end). After fork(), the parent typically writes to pipefd[1] and the child reads from pipefd[0]. Unused ends must be closed to avoid blocking. Pipes are synchronised — a read blocks until data is available.

Pipe Diagram

```
  PARENT                      CHILD
  ------                      -----
  close(pipefd[0])            close(pipefd[1])
       |                           |
  write(pipefd[1], msg)       read(pipefd[0], buf)
       |                           |
  +-----------+               +-----------+
  | pipefd[1] | ==========> | pipefd[0] |
  | (write)   |    PIPE     | (read)    |
  +-----------+               +-----------+
```

Algorithm

- Create a pipe using pipe(pipefd).
- Call fork() to create a child process.
- Parent: close read end, write message to write end, close write end, call wait().
- Child: close write end, read from read end, print message, close read end, exit.
- Stop.

C Program

```
/* pipe_ipc.c - IPC using pipes */
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <string.h>
#include <sys/wait.h>

int main() {
    int pipefd[2];   /* pipefd[0]=read end, pipefd[1]=write end */
    pid_t pid;
    char write_msg[] = "Hello from Parent via Pipe!";
    char read_buf[128];
    ssize_t bytes;

    /* Create the pipe */
    if (pipe(pipefd) == -1) {
        perror("pipe creation failed");
        exit(EXIT_FAILURE);
    }

    pid = fork();
```

```
    if (pid < 0) {
        perror("fork failed");
        exit(EXIT_FAILURE);

    } else if (pid == 0) {
        /* Child process: READER */
        close(pipefd[1]);   /* Close unused write end */

        bytes = read(pipefd[0], read_buf, sizeof(read_buf) - 1);
        read_buf[bytes] = '\0';
        printf("[Child]  PID=%d | Received: \"%s\"\n",
               getpid(), read_buf);

        close(pipefd[0]);
        exit(EXIT_SUCCESS);

    } else {
        /* Parent process: WRITER */
        close(pipefd[0]);   /* Close unused read end */

        write(pipefd[1], write_msg, strlen(write_msg));
        printf("[Parent] PID=%d | Sent:     \"%s\"\n",
               getpid(), write_msg);

        close(pipefd[1]);
        wait(NULL);
        printf("[Parent] Child process completed.\n");
    }

    return 0;
}
```

Code Explanation

pipe() creates the two FDs. After fork(), the parent closes pipefd[0] and writes to pipefd[1]. The child closes pipefd[1] and reads from pipefd[0]. Closing unused ends is critical — an open write end on the child side would prevent the child's read() from returning on EOF. wait() prevents the parent from exiting before the child.

Compilation

```
student@ubuntu:~$ gcc pipe_ipc.c -o pipe_ipc
```

Execution

```
student@ubuntu:~$ ./pipe_ipc
```

Sample Output

```
student@ubuntu:~$ ./pipe_ipc
[Parent] PID=6021 | Sent:     "Hello from Parent via Pipe!"
[Child]  PID=6022 | Received: "Hello from Parent via Pipe!"
[Parent] Child process completed.
student@ubuntu:~$
```

Result: The C program successfully implemented inter-process communication between parent and child processes using a unidirectional UNIX pipe.

Experiment 6

Question

Simulate the following CPU scheduling algorithms: a. Round Robin b. SJF (Shortest Job First)

Theory

Round Robin (RR) allocates the CPU to each ready process in a cyclic order for a fixed time quantum. It provides fair scheduling and is suitable for time-sharing systems. SJF (non-preemptive) selects the process with the smallest burst time from the ready queue at the time of scheduling. SJF minimises average waiting time but can cause starvation of longer processes.

Part a – Round Robin Scheduling

C Program

```
/* round_robin.c - Round Robin CPU Scheduling Simulation */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define MAX 10

struct Process {
    int pid, arrival, burst, remaining;
    int completion, waiting, turnaround;
};

int main() {
    struct Process p[MAX];
    int n, quantum, time = 0, completed = 0;
    float avg_wt = 0, avg_tat = 0;

    printf("=== Round Robin CPU Scheduling ===\n");
    printf("Enter number of processes: ");
    scanf("%d", &n);
    printf("Enter time quantum: ");
    scanf("%d", &quantum);

    for (int i = 0; i < n; i++) {
        p[i].pid = i + 1;
        printf("P%d Arrival Time: ", i + 1); scanf("%d",
&p[i].arrival);
        printf("P%d Burst Time  : ", i + 1); scanf("%d", &p[i].burst);
        p[i].remaining = p[i].burst;
    }

    printf("\nGantt Chart:\n");
    printf("%-8s %s\n", "Time", "Process");

    while (completed < n) {
        int found = 0;
        for (int i = 0; i < n; i++) {
            if (p[i].remaining > 0 && p[i].arrival <= time) {
                found = 1;
                int exec = (p[i].remaining < quantum) ?
                           p[i].remaining : quantum;
```

```
                printf("[%2d - %2d]  P%d\n", time, time + exec,
p[i].pid);
                time += exec;
                p[i].remaining -= exec;
                if (p[i].remaining == 0) {
                    p[i].completion  = time;
                    p[i].turnaround  = time - p[i].arrival;
                    p[i].waiting     = p[i].turnaround - p[i].burst;
                    completed++;
                }
            }
        }
        if (!found) time++;
    }

    printf("\n%-4s %-8s %-8s %-12s %-10s %-12s\n",

"PID","Arrival","Burst","Completion","Waiting","Turnaround");
    for (int i = 0; i < n; i++) {
        printf("P%-3d %-8d %-8d %-12d %-10d %-12d\n",
               p[i].pid, p[i].arrival, p[i].burst,
               p[i].completion, p[i].waiting, p[i].turnaround);
        avg_wt  += p[i].waiting;
        avg_tat += p[i].turnaround;
    }
    printf("\nAverage Waiting Time    : %.2f\n", avg_wt  / n);
    printf("Average Turnaround Time : %.2f\n", avg_tat / n);
    return 0;
}
```

Compilation & Execution — Round Robin

```
student@ubuntu:~$ gcc round_robin.c -o round_robin
student@ubuntu:~$ ./round_robin
```

Sample Output — Round Robin

```
student@ubuntu:~$ ./round_robin
=== Round Robin CPU Scheduling ===
Enter number of processes: 3
Enter time quantum: 2
P1 Arrival Time: 0
P1 Burst Time  : 5
P2 Arrival Time: 1
P2 Burst Time  : 3
P3 Arrival Time: 2
P3 Burst Time  : 4

Gantt Chart:
Time     Process
[ 0 -  2]  P1
[ 2 -  4]  P2
[ 4 -  6]  P3
[ 6 -  8]  P1
[ 8 -  9]  P2
[ 9 - 11]  P3
[11 - 12]  P1
```

```
PID  Arrival  Burst   Completion  Waiting    Turnaround
P1   0        5       12          7          12
P2   1        3       9           5          8
P3   2        4       11          5          9

Average Waiting Time    : 5.67
Average Turnaround Time : 9.67
student@ubuntu:~$
```

Part b – Shortest Job First (SJF) Scheduling

C Program

```
/* sjf.c - Shortest Job First (Non-Preemptive) CPU Scheduling */
#include <stdio.h>
#include <stdlib.h>

#define MAX 10
#define INF 9999

struct Process {
    int pid, arrival, burst;
    int completion, waiting, turnaround;
    int done;
};

int main() {
    struct Process p[MAX];
    int n, time = 0, completed = 0;
    float avg_wt = 0, avg_tat = 0;

    printf("=== SJF (Non-Preemptive) Scheduling ===\n");
    printf("Enter number of processes: ");
    scanf("%d", &n);

    for (int i = 0; i < n; i++) {
        p[i].pid = i + 1; p[i].done = 0;
        printf("P%d Arrival Time: ", i+1); scanf("%d", &p[i].arrival);
        printf("P%d Burst Time  : ", i+1); scanf("%d", &p[i].burst);
    }

    printf("\nGantt Chart:\n");
    printf("%-8s %s\n", "Time", "Process");

    while (completed < n) {
        int shortest = -1, min_burst = INF;
        for (int i = 0; i < n; i++) {
            if (!p[i].done && p[i].arrival <= time &&
                p[i].burst < min_burst) {
                min_burst = p[i].burst;
                shortest  = i;
            }
        }
        if (shortest == -1) { time++; continue; }

        printf("[%2d - %2d]  P%d\n",
               time, time + p[shortest].burst, p[shortest].pid);
        time              += p[shortest].burst;
```

```
        p[shortest].completion  = time;
        p[shortest].turnaround  = time - p[shortest].arrival;
        p[shortest].waiting     = p[shortest].turnaround -
p[shortest].burst;
        p[shortest].done        = 1;
        completed++;
    }

    printf("\n%-4s %-8s %-8s %-12s %-10s %-12s\n",

"PID","Arrival","Burst","Completion","Waiting","Turnaround");
    for (int i = 0; i < n; i++) {
        printf("P%-3d %-8d %-8d %-12d %-10d %-12d\n",
               p[i].pid, p[i].arrival, p[i].burst,
               p[i].completion, p[i].waiting, p[i].turnaround);
        avg_wt  += p[i].waiting;
        avg_tat += p[i].turnaround;
    }
    printf("\nAverage Waiting Time    : %.2f\n", avg_wt  / n);
    printf("Average Turnaround Time : %.2f\n", avg_tat / n);
    return 0;
}
```

Compilation & Execution — SJF

```
student@ubuntu:~$ gcc sjf.c -o sjf
student@ubuntu:~$ ./sjf
```

Sample Output — SJF

```
student@ubuntu:~$ ./sjf
=== SJF (Non-Preemptive) Scheduling ===
Enter number of processes: 3
P1 Arrival Time: 0
P1 Burst Time  : 6
P2 Arrival Time: 1
P2 Burst Time  : 2
P3 Arrival Time: 2
P3 Burst Time  : 4

Gantt Chart:
Time     Process
[ 0 -  6]  P1
[ 6 -  8]  P2
[ 8 - 12]  P3

PID  Arrival  Burst   Completion  Waiting    Turnaround
P1   0        6       6           0          6
P2   1        2       8           5          7
P3   2        4       12          6          10

Average Waiting Time    : 3.67
Average Turnaround Time : 7.67
student@ubuntu:~$
```

Result: Both CPU scheduling algorithms (Round Robin with time quantum and non-preemptive SJF) were simulated successfully, producing accurate Gantt charts and performance metrics.

Experiment 7

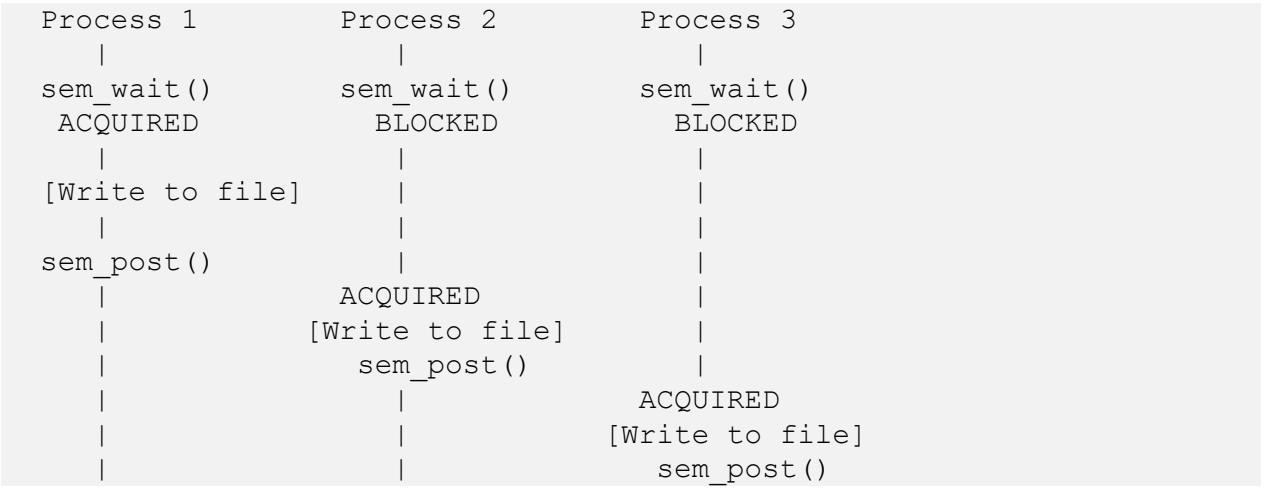
Question

Write a program that illustrates file locking using semaphores.

Theory

Semaphores are synchronisation primitives used to control access to shared resources. POSIX named semaphores are created with sem_open(). sem_wait() decrements the semaphore (acquires the lock); sem_post() increments it (releases the lock). A binary semaphore (initial value 1) acts as a mutex, ensuring that only one process writes to a shared file at a time.

Semaphore Locking Diagram



Algorithm

- Create a named binary semaphore (initial value = 1) using sem_open().
- Fork three child processes.
- Each child calls sem_wait() to acquire the lock.
- Write to the shared file inside the critical section.
- Call sem_post() to release the lock.
- Parent waits for all children then reads the file.
- Remove the semaphore with sem_unlink().

C Program

```
/* file_lock_sem.c - File locking using POSIX named semaphores */
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <semaphore.h>
#include <fcntl.h>
#include <sys/wait.h>
#include <string.h>

#define SEM_NAME  "/file_lock_sem"
#define LOCK_FILE "shared_file.txt"

void write_file(int proc_id) {
    sem_t *sem = sem_open(SEM_NAME, 0);
    if (sem == SEM_FAILED) { perror("sem_open"); exit(1); }
```

```
    printf("[Process %d] Waiting for file lock...\n", proc_id);
    sem_wait(sem);   /* Acquire lock */
    printf("[Process %d] Lock acquired. Writing...\n", proc_id);

    int fd = open(LOCK_FILE, O_WRONLY | O_APPEND | O_CREAT, 0644);
    if (fd == -1) { perror("open"); sem_post(sem); exit(1); }

    char buf[64];
    snprintf(buf, sizeof(buf), "Process %d wrote this line.\n",
proc_id);
    write(fd, buf, strlen(buf));
    sleep(1);   /* Simulate file operation */
    close(fd);

    printf("[Process %d] Done. Releasing lock.\n", proc_id);
    sem_post(sem);   /* Release lock */
    sem_close(sem);
}

int main() {
    /* Create binary semaphore (value=1) */
    sem_t *sem = sem_open(SEM_NAME, O_CREAT | O_EXCL, 0644, 1);
    if (sem == SEM_FAILED) {
        sem_unlink(SEM_NAME);
        sem = sem_open(SEM_NAME, O_CREAT, 0644, 1);
    }
    sem_close(sem);

    printf("=== File Locking using Semaphores ===\n\n");

    for (int i = 1; i <= 3; i++) {
        pid_t pid = fork();
        if (pid == 0) {
            write_file(i);
            exit(EXIT_SUCCESS);
        }
    }

    for (int i = 0; i < 3; i++) wait(NULL);

    printf("\nFinal contents of %s:\n", LOCK_FILE);
    printf("==============================\n");
    int fd = open(LOCK_FILE, O_RDONLY);
    if (fd != -1) {
        char buf[256]; ssize_t n;
        while ((n = read(fd, buf, sizeof(buf)-1)) > 0) {
            buf[n] = '\0'; printf("%s", buf);
        }
        close(fd);
    }

    sem_unlink(SEM_NAME);
    remove(LOCK_FILE);
    return 0;
}
```

Code Explanation

sem_open() creates a named semaphore with initial value 1 (binary). Each forked child opens the semaphore and calls sem_wait() to acquire it. The critical section opens the shared file in append mode and writes to it. sem_post() releases the semaphore, allowing the next blocked process to proceed. This serialises file access.

Compilation

```
student@ubuntu:~$ gcc file_lock_sem.c -o file_lock_sem -lpthread
```

Execution

```
student@ubuntu:~$ ./file_lock_sem
```

Sample Output

```
student@ubuntu:~$ ./file_lock_sem
=== File Locking using Semaphores ===

[Process 1] Waiting for file lock...
[Process 1] Lock acquired. Writing...
[Process 2] Waiting for file lock...
[Process 3] Waiting for file lock...
[Process 1] Done. Releasing lock.
[Process 2] Lock acquired. Writing...
[Process 2] Done. Releasing lock.
[Process 3] Lock acquired. Writing...
[Process 3] Done. Releasing lock.

Final contents of shared_file.txt:
==================================
Process 1 wrote this line.
Process 2 wrote this line.
Process 3 wrote this line.
student@ubuntu:~$
```

Result: The C program successfully demonstrated exclusive file locking using POSIX named semaphores, preventing concurrent writes to a shared file.

Experiment 8

Question

Write a program that implements a producer-consumer system with two processes (using semaphores).

Theory

The Producer-Consumer problem uses a bounded buffer shared between two processes. Three semaphores are required: empty (tracks free slots, initial = buffer_size), full (tracks filled slots, initial = 0), and mutex (binary semaphore for mutual exclusion, initial = 1). The producer waits on empty and signals full; the consumer waits on full and signals empty. Shared memory is used to share the buffer between processes.

Producer-Consumer Diagram

```
PRODUCER                    BUFFER                    CONSUMER
--------                    ------                    --------
sem_wait(empty)         [  ,  ,  ,  ,  ]         sem_wait(full)
sem_wait(mutex) -----> [add item      ] <----- sem_wait(mutex)
sem_post(mutex)         [  ,  ,  ,  ,  ]         sem_post(mutex)
sem_post(full)                                   sem_post(empty)

Semaphores: empty(5), full(0), mutex(1)  [buffer size = 5]
```

Algorithm

- Create shared memory for the circular buffer structure.
- Initialise semaphores: empty=BUFFER_SIZE, full=0, mutex=1.
- Fork: child is Consumer, parent is Producer.
- Producer: sem_wait(empty) -> sem_wait(mutex) -> produce -> sem_post(mutex) -> sem_post(full).
- Consumer: sem_wait(full) -> sem_wait(mutex) -> consume -> sem_post(mutex) -> sem_post(empty).
- Wait for child, then clean up semaphores and shared memory.

C Program

```
/* prod_cons.c - Producer-Consumer using semaphores and shared memory
*/
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <semaphore.h>
#include <fcntl.h>
#include <sys/mman.h>
#include <sys/wait.h>

#define BUFFER_SIZE   5
#define NUM_ITEMS     8
#define SEM_EMPTY    "/pc_empty"
#define SEM_FULL     "/pc_full"
#define SEM_MUTEX    "/pc_mutex"
#define SHM_NAME     "/pc_shm"

typedef struct {
    int buffer[BUFFER_SIZE];
    int in, out, count;
```

```
} SharedBuf;

int main() {
    /* Create shared memory */
    int shm_fd = shm_open(SHM_NAME, O_CREAT | O_RDWR, 0644);
    ftruncate(shm_fd, sizeof(SharedBuf));
    SharedBuf *sb = mmap(NULL, sizeof(SharedBuf),
                         PROT_READ | PROT_WRITE, MAP_SHARED, shm_fd,
0);
    sb->in = sb->out = sb->count = 0;

    /* Initialise semaphores */
    sem_unlink(SEM_EMPTY); sem_unlink(SEM_FULL); sem_unlink(SEM_MUTEX);
    sem_t *sem_e = sem_open(SEM_EMPTY, O_CREAT, 0644, BUFFER_SIZE);
    sem_t *sem_f = sem_open(SEM_FULL,  O_CREAT, 0644, 0);
    sem_t *sem_m = sem_open(SEM_MUTEX, O_CREAT, 0644, 1);

    printf("=== Producer-Consumer (Semaphores) ===\n");
    printf("Buffer Size: %d  |  Items: %d\n\n", BUFFER_SIZE,
NUM_ITEMS);

    pid_t pid = fork();

    if (pid == 0) {
        /* CONSUMER */
        for (int i = 0; i < NUM_ITEMS; i++) {
            sem_wait(sem_f);
            sem_wait(sem_m);
            int item = sb->buffer[sb->out];
            sb->out = (sb->out + 1) % BUFFER_SIZE;
            sb->count--;
            printf("[Consumer] Consumed: %2d  | Buffer: %d\n",
                   item, sb->count);
            sem_post(sem_m);
            sem_post(sem_e);
            sleep(1);
        }
        exit(EXIT_SUCCESS);

    } else {
        /* PRODUCER */
        for (int i = 1; i <= NUM_ITEMS; i++) {
            sem_wait(sem_e);
            sem_wait(sem_m);
            sb->buffer[sb->in] = i;
            sb->in = (sb->in + 1) % BUFFER_SIZE;
            sb->count++;
            printf("[Producer] Produced: %2d  | Buffer: %d\n",
                   i, sb->count);
            sem_post(sem_m);
            sem_post(sem_f);
            sleep(1);
        }
        wait(NULL);

        sem_close(sem_e); sem_unlink(SEM_EMPTY);
        sem_close(sem_f); sem_unlink(SEM_FULL);
```

```
        sem_close(sem_m); sem_unlink(SEM_MUTEX);
        munmap(sb, sizeof(SharedBuf));
        shm_unlink(SHM_NAME);
        printf("\nAll items produced and consumed.\n");
    }
    return 0;
}
```

Code Explanation

shm_open() and mmap() create a shared circular buffer. Three POSIX named semaphores synchronise access. The Producer calls sem_wait(empty) to check for a free slot, sem_wait(mutex) for mutual exclusion, places the item, then signals sem_post(mutex) and sem_post(full). The Consumer reverses this sequence, consuming from the head.

Compilation

```
student@ubuntu:~$ gcc prod_cons.c -o prod_cons -lpthread
```

Execution

```
student@ubuntu:~$ ./prod_cons
```

Sample Output

```
student@ubuntu:~$ ./prod_cons
=== Producer-Consumer (Semaphores) ===
Buffer Size: 5  |  Items: 8

[Producer] Produced:  1  | Buffer: 1
[Consumer] Consumed:  1  | Buffer: 0
[Producer] Produced:  2  | Buffer: 1
[Consumer] Consumed:  2  | Buffer: 0
[Producer] Produced:  3  | Buffer: 1
[Consumer] Consumed:  3  | Buffer: 0
[Producer] Produced:  4  | Buffer: 1
[Consumer] Consumed:  4  | Buffer: 0
[Producer] Produced:  5  | Buffer: 1
[Consumer] Consumed:  5  | Buffer: 0
[Producer] Produced:  6  | Buffer: 1
[Consumer] Consumed:  6  | Buffer: 0
[Producer] Produced:  7  | Buffer: 1
[Consumer] Consumed:  7  | Buffer: 0
[Producer] Produced:  8  | Buffer: 1
[Consumer] Consumed:  8  | Buffer: 0

All items produced and consumed.
student@ubuntu:~$
```

Result: The C program successfully implemented the Producer-Consumer problem with two processes using POSIX named semaphores and shared memory for the bounded buffer.

Experiment 9

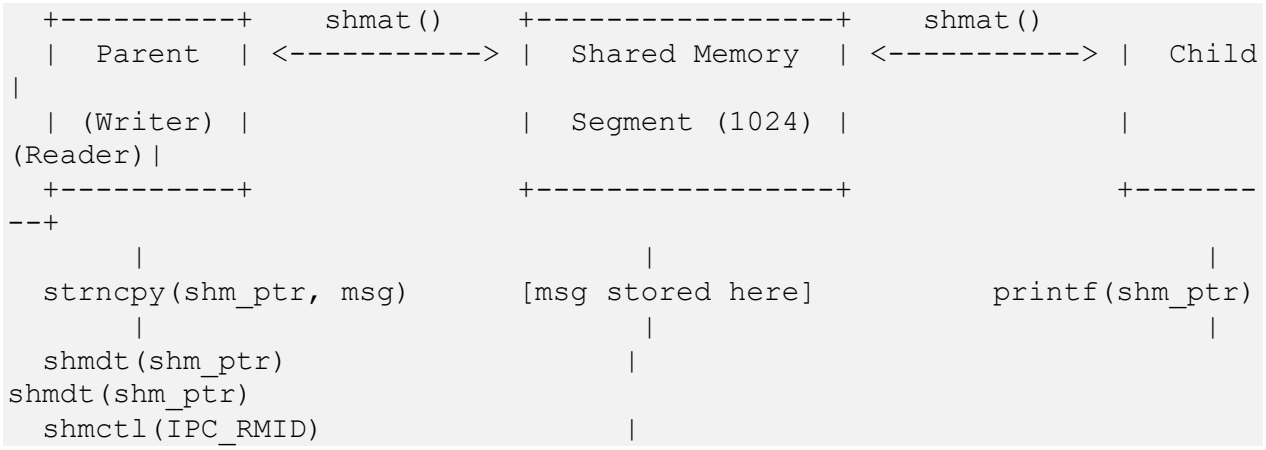
Question

Write a program that illustrates inter process communication using shared memory system calls.

Theory

Shared memory is the fastest form of IPC in UNIX. shmget() creates or accesses a shared memory segment identified by a key. shmat() attaches the segment to the process address space and returns a pointer. shmdt() detaches the segment. shmctl() with IPC_RMID deletes the segment. Communication occurs through direct memory read/write operations on the shared region.

Shared Memory Diagram



Algorithm

- Create shared memory using shmget() with IPC_CREAT flag.
- Fork a child process.
- Parent: attach with shmat(), write message, detach, wait for child.
- Child: sleep briefly, attach with shmat(), read and print message, detach.
- Parent: delete shared memory with shmctl(IPC_RMID).
- Stop.

C Program

```
/* shared_mem.c - IPC using System V Shared Memory */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/ipc.h>
#include <sys/shm.h>
#include <sys/wait.h>
#include <unistd.h>

#define SHM_KEY  2345
#define SHM_SIZE 1024

int main() {
    int shmid;
    char *shm_ptr;

    /* Create shared memory segment */
    shmid = shmget(SHM_KEY, SHM_SIZE, IPC_CREAT | 0666);
```

```
    if (shmid == -1) {
        perror("shmget failed");
        exit(EXIT_FAILURE);
    }
    printf("Shared memory segment created. SHMID: %d\n", shmid);

    pid_t pid = fork();

    if (pid < 0) {
        perror("fork failed");
        exit(EXIT_FAILURE);

    } else if (pid == 0) {
        /* Child: READER */
        sleep(1);   /* Wait for parent to write */

        shm_ptr = (char *)shmat(shmid, NULL, 0);
        if (shm_ptr == (char *)-1) {
            perror("shmat (child) failed");
            exit(EXIT_FAILURE);
        }

        printf("[Child]  PID=%d | Read from shared memory: \"%s\"\n",
               getpid(), shm_ptr);

        shmdt(shm_ptr);
        exit(EXIT_SUCCESS);

    } else {
        /* Parent: WRITER */
        shm_ptr = (char *)shmat(shmid, NULL, 0);
        if (shm_ptr == (char *)-1) {
            perror("shmat (parent) failed");
            exit(EXIT_FAILURE);
        }

        const char *message = "Hello from Parent via Shared Memory!";
        strncpy(shm_ptr, message, SHM_SIZE - 1);
        printf("[Parent] PID=%d | Written to shared memory: \"%s\"\n",
               getpid(), message);

        wait(NULL);   /* Wait for child */

        shmdt(shm_ptr);
        shmctl(shmid, IPC_RMID, NULL);   /* Delete segment */
        printf("Shared memory segment removed.\n");
    }

    return 0;
}
```

Code Explanation

shmget() with key SHM_KEY and IPC_CREAT creates a 1024-byte shared segment. shmat() attaches it to the process's virtual address space. The parent writes using strncpy() directly to the pointer. The child sleeps 1 second to allow the parent to write, then attaches the same segment and reads. shmctl(IPC_RMID) removes the segment after use.

Compilation

```
student@ubuntu:~$ gcc shared_mem.c -o shared_mem
```

Execution

```
student@ubuntu:~$ ./shared_mem
```

Sample Output

```
student@ubuntu:~$ ./shared_mem
Shared memory segment created. SHMID: 196608
[Parent] PID=7201 | Written to shared memory: "Hello from Parent via
Shared Memory!"
[Child]  PID=7202 | Read from shared memory: "Hello from Parent via
Shared Memory!"
Shared memory segment removed.
student@ubuntu:~$
```

Result: The C program successfully demonstrated IPC using System V shared memory, with the parent writing a message and the child reading it from the shared segment.

Experiment 10

Question

Write a program that illustrates the following: a. Creating message queue. b. Writing to a message queue. c. Reading from a message queue.

Theory

System V message queues allow processes to exchange typed messages asynchronously. msgget() creates or accesses a queue identified by a key. msgsnd() appends a message to the queue. msgrcv() reads a message of a specified type from the queue. msgctl() with IPC_RMID removes the queue. Each message has a type field (long) and a text payload.

Message Queue Diagram

```
SENDER                  MESSAGE QUEUE                   RECEIVER
------                  -------------                   --------
                    +----+----+----+----+
msgsnd() --------> |msg1|msg2|... |    |  <-------- msgrcv()
                    +----+----+----+----+
                    type=1  type=2

msgget(KEY, IPC_CREAT)  --> Creates the queue
msgsnd(msgid, &msg, size, 0)  --> Sends message
msgrcv(msgid, &msg, size, type, 0)  --> Receives by type
msgctl(msgid, IPC_RMID, NULL)  --> Deletes queue
```

Algorithm

- Create a message queue using msgget() with IPC_CREAT and key.
- Prepare a message struct with msg_type and msg_text fields.
- Send the message to the queue using msgsnd().
- Receive the message from the queue using msgrcv() with matching type.
- Print the received message.
- Remove the message queue using msgctl(IPC_RMID).
- Stop.

C Program

```c
/* msg_queue.c - Message Queue: Create, Write, Read */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/ipc.h>
#include <sys/msg.h>
#include <unistd.h>

#define MSG_KEY  6789
#define MSG_SIZE 128

/* Message structure - must start with long msg_type */
struct MsgBuf {
    long msg_type;
    char msg_text[MSG_SIZE];
};

int main() {
```

```
    int msgid;
    struct MsgBuf send_msg, recv_msg;

    printf("=== Message Queue Demonstration ===\n\n");

    /* ---------------------------------------- *
     * a. Create Message Queue                  *
     * ---------------------------------------- */
    msgid = msgget(MSG_KEY, IPC_CREAT | 0666);
    if (msgid == -1) {
        perror("msgget failed");
        exit(EXIT_FAILURE);
    }
    printf("a. Message Queue Created. MSGID: %d\n\n", msgid);

    /* ---------------------------------------- *
     * b. Write to Message Queue                *
     * ---------------------------------------- */
    send_msg.msg_type = 1;
    strncpy(send_msg.msg_text, "Hello via Message Queue!", MSG_SIZE -
1);

    if (msgsnd(msgid, &send_msg, sizeof(send_msg.msg_text), 0) == -1) {
        perror("msgsnd failed");
        msgctl(msgid, IPC_RMID, NULL);
        exit(EXIT_FAILURE);
    }
    printf("b. Message Sent (type=%ld): \"%s\"\n\n",
           send_msg.msg_type, send_msg.msg_text);

    /* Send a second message of type 2 */
    send_msg.msg_type = 2;
    strncpy(send_msg.msg_text, "Second message (type 2)!", MSG_SIZE -
1);
    msgsnd(msgid, &send_msg, sizeof(send_msg.msg_text), 0);
    printf("   Message Sent (type=%ld): \"%s\"\n\n",
           send_msg.msg_type, send_msg.msg_text);

    /* ---------------------------------------- *
     * c. Read from Message Queue               *
     * ---------------------------------------- */
    /* Receive message of type 1 */
    memset(&recv_msg, 0, sizeof(recv_msg));
    if (msgrcv(msgid, &recv_msg, sizeof(recv_msg.msg_text), 1, 0) == -
1) {
        perror("msgrcv failed");
        msgctl(msgid, IPC_RMID, NULL);
        exit(EXIT_FAILURE);
    }
    printf("c. Message Received (type=1): \"%s\"\n",
recv_msg.msg_text);

    /* Receive message of type 2 */
    memset(&recv_msg, 0, sizeof(recv_msg));
    msgrcv(msgid, &recv_msg, sizeof(recv_msg.msg_text), 2, 0);
    printf("   Message Received (type=2): \"%s\"\n\n",
recv_msg.msg_text);
```

```
    /* Remove message queue */
    if (msgctl(msgid, IPC_RMID, NULL) == -1)
        perror("msgctl failed");
    else
        printf("Message Queue removed successfully.\n");

    return 0;
}
```

Code Explanation

msgget() with IPC_CREAT creates the queue with permissions 0666. struct MsgBuf has a mandatory long msg_type field followed by the payload. msgsnd() appends the message; the size argument is the payload size, not the total struct size. msgrcv() with type=1 retrieves only type-1 messages. msgctl(IPC_RMID) removes the queue and all its messages.

Compilation

```
student@ubuntu:~$ gcc msg_queue.c -o msg_queue
```

Execution

```
student@ubuntu:~$ ./msg_queue
```

Sample Output

```
student@ubuntu:~$ ./msg_queue
=== Message Queue Demonstration ===

a. Message Queue Created. MSGID: 32768

b. Message Sent (type=1): "Hello via Message Queue!"

   Message Sent (type=2): "Second message (type 2)!"

c. Message Received (type=1): "Hello via Message Queue!"
   Message Received (type=2): "Second message (type 2)!"

Message Queue removed successfully.
student@ubuntu:~$
```

Result: The C program successfully demonstrated all three message queue operations: creating a queue with msgget(), writing messages with msgsnd(), and reading messages with msgrcv().